Distributed Algorithms

Vania Marangozova-Martin

Motivation

- **In the last lecture we saw that coordination is the complex part in managing a distributed system**
  - It is about how to control the way entities interact
  - it is about deciding what they say to each other
    - Remember? In a distributed system there is no shared memory, so the only way to communicate is through messages over the network
  - it is about making them work together towards the commun goal

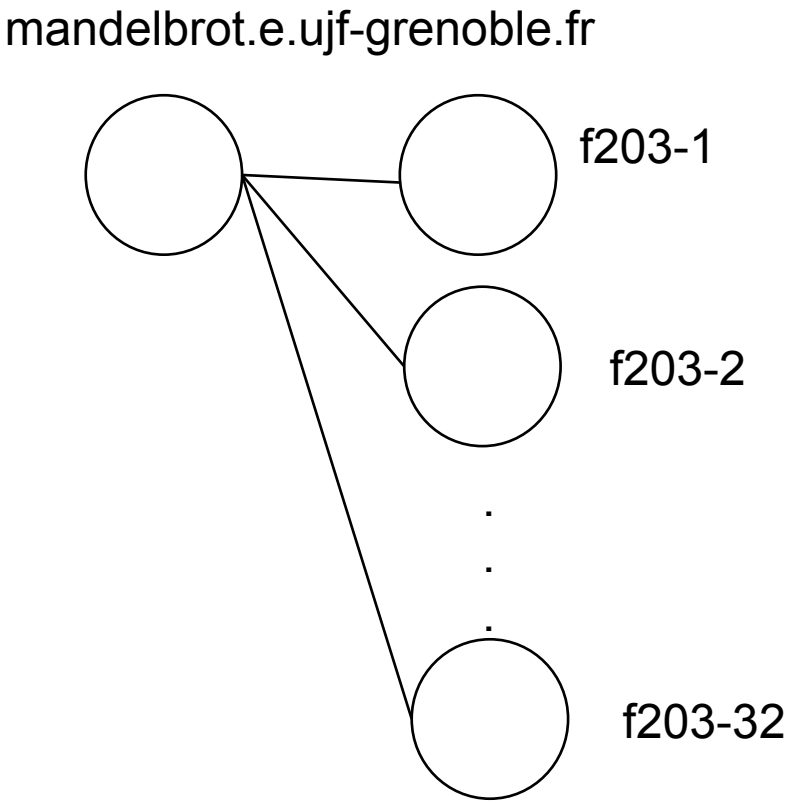
In this lecture (and certainly the next one)

- **Explicit the system model**
- **Introduce the way we will write distributed algorithms**
  - define the syntax and semantics
- **Start some basic algorithms**
  - wave algorithms
  - election algorithms

Distributed system model

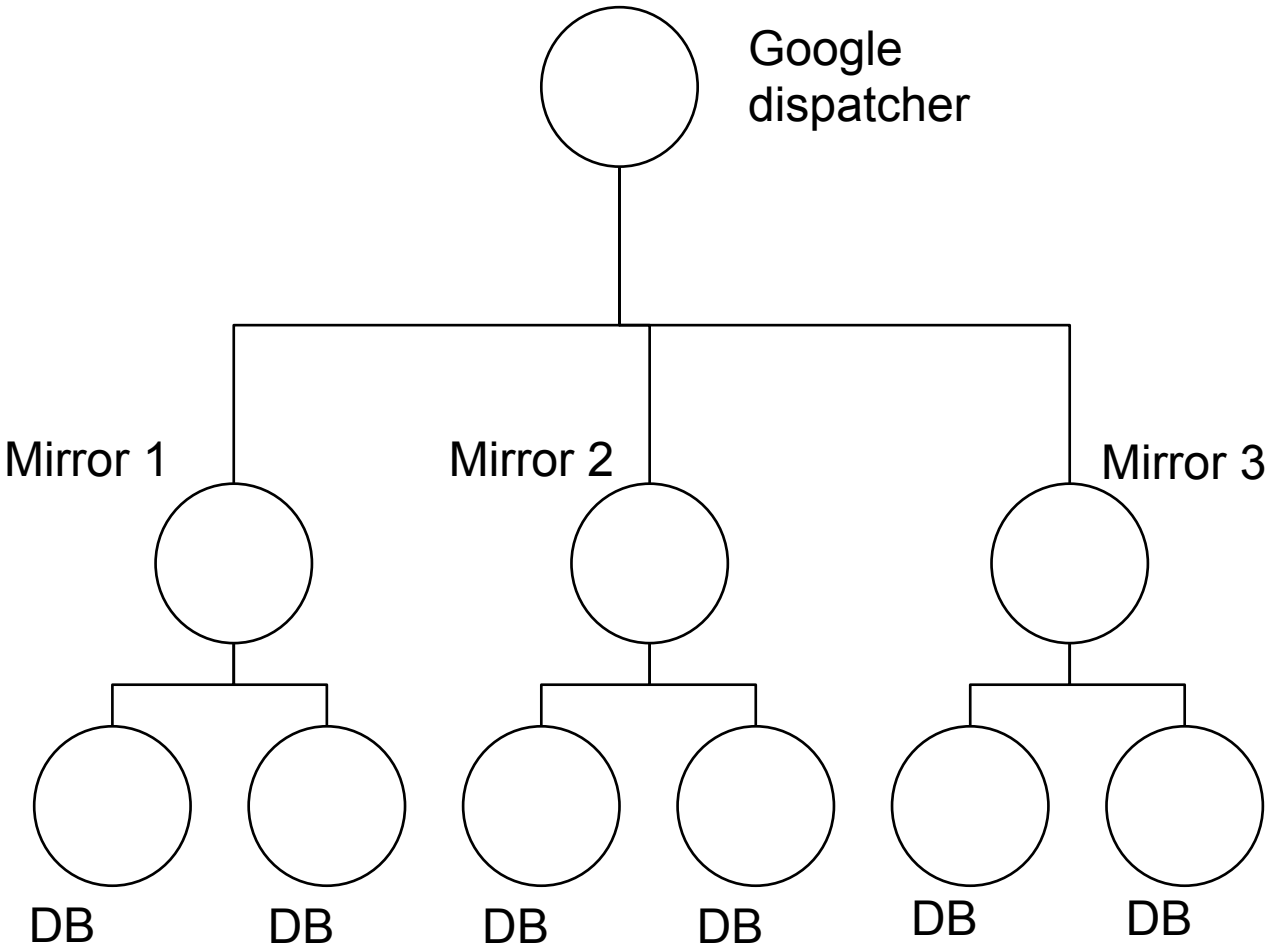
- **A system is represented by a graph**
  - nodes and links
  - A node is an abstraction of the entity participating in the system
    - a physical machine
    - a virtual machine
    - a process
    - an application
    - a service
    - ...
  - a link is an abstraction of the communication channel between two entities
    - physical link
    - virtual link

Example : physical machines



The school

Example : servers



In our model

- **The nodes have the same role**
  - P2P (peer to peer) model = each node has the same functionality
  - each node executes the same algorithm
  - each node has the same local data
  - nodes have unique IDs (if not anonymous)
- **The system is asynchronous**
  - the nodes advance at their proper specific speed
  - we do not know when things happen (when a node does something/ executes an operation)
  - a synchronous system is when all entities execute one step at the same time
  - anyway, there is no global time
- **The links are reliable**
  - message transmission takes a finite time
  - we cannot put an upper limit to this time (no TIMEOUT)

Node operations

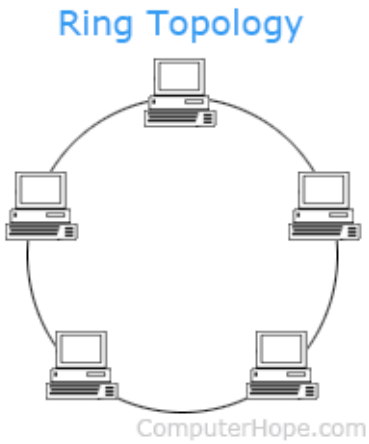
- **Nodes have their local data**
- **What they can do is**
  - do something with their local data
  - send a message to a node/channel
  - receive a message from a node/channel

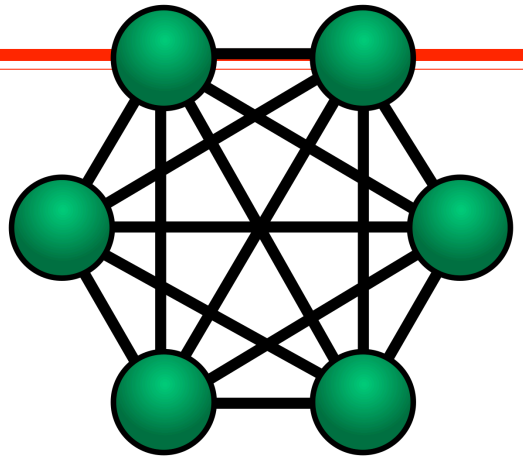
  Depending on the system, sometimes a node knows its comm. channels but does not know to/form which node they connect and vice versa

- **Message send is non blocking**
- **Message receive is blocking**
- **Hypothesis : message transmission buffers are infinite**

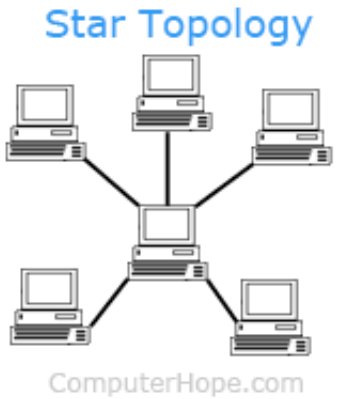
Well-known topologies

- **Ring**

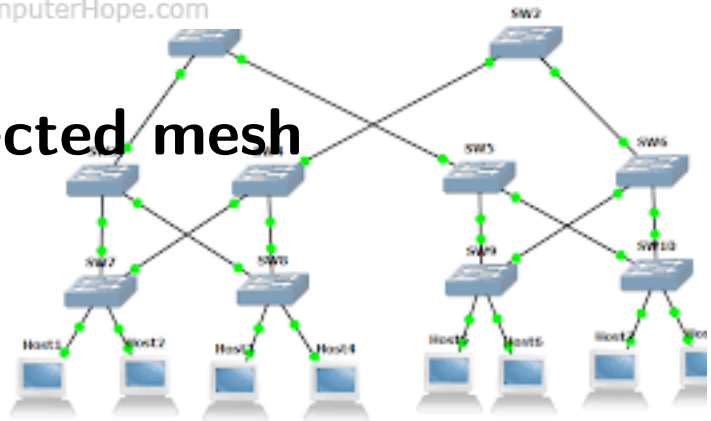




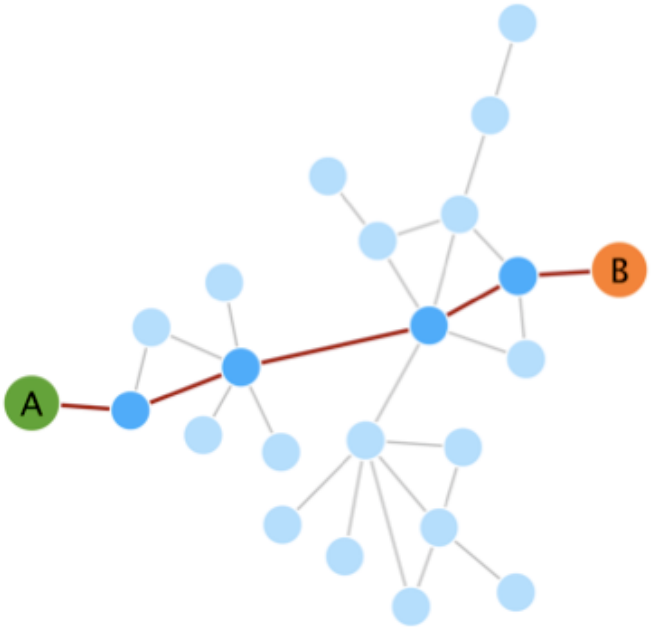
- **Star**



- **Fully-connected mesh**



- **Tree**



- **Graph**

Distributed algorithms = protocols

- **What we will be interested in is the exchanges of messages between nodes**
- **this is about the PROTOCOL**
  - what types of messages
  - what sequences of messages
  - what is the treatment of messages

- **Nodes exchange in order to**
  - broadcast information
  - compute the state of the system
  - decide something
  - detect some interesting property (termination, deadlock)

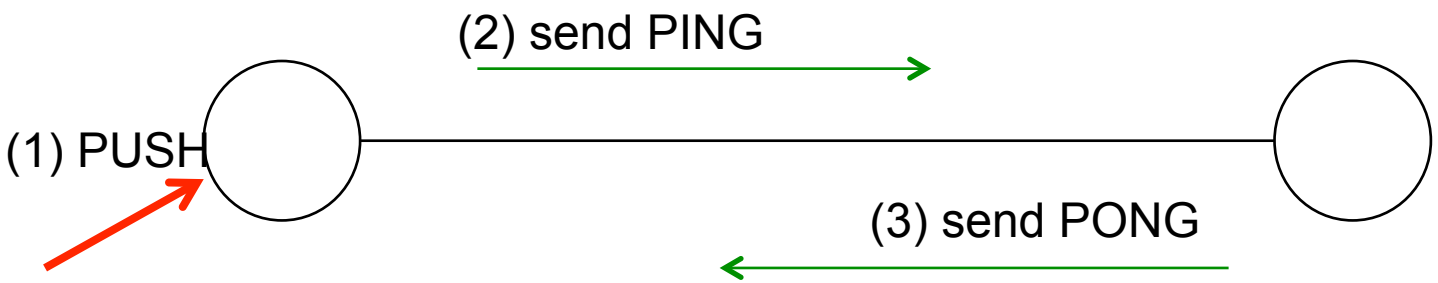
To write algorithms we will use rules

- **Trigger -> réaction**
- **The trigger is related to the change of the state of the node**
  - reception of a message
  - change in the data of the node
- **The reaction**
  - mey involve the sending of one or multiple messages
  - the change of the state (value) of the local data

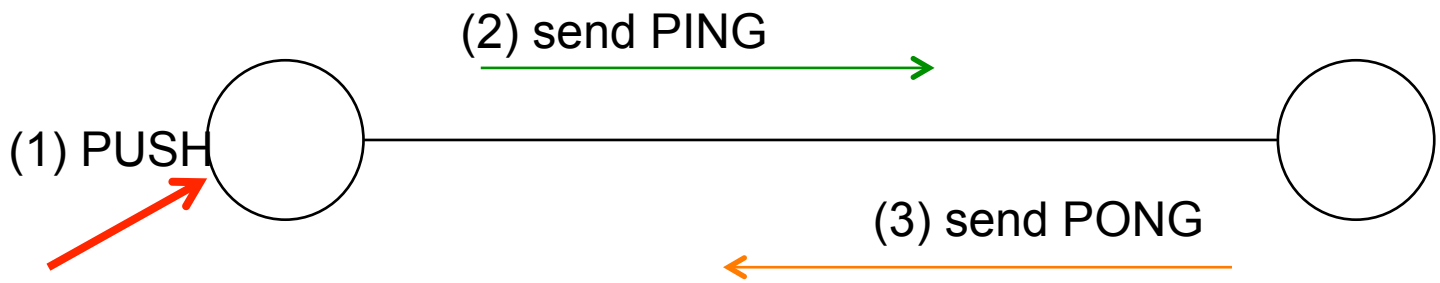
- **We consider that the rules are atomic**
- **We will strive to write rules that have different triggers, otherwise the system will be non deterministic**

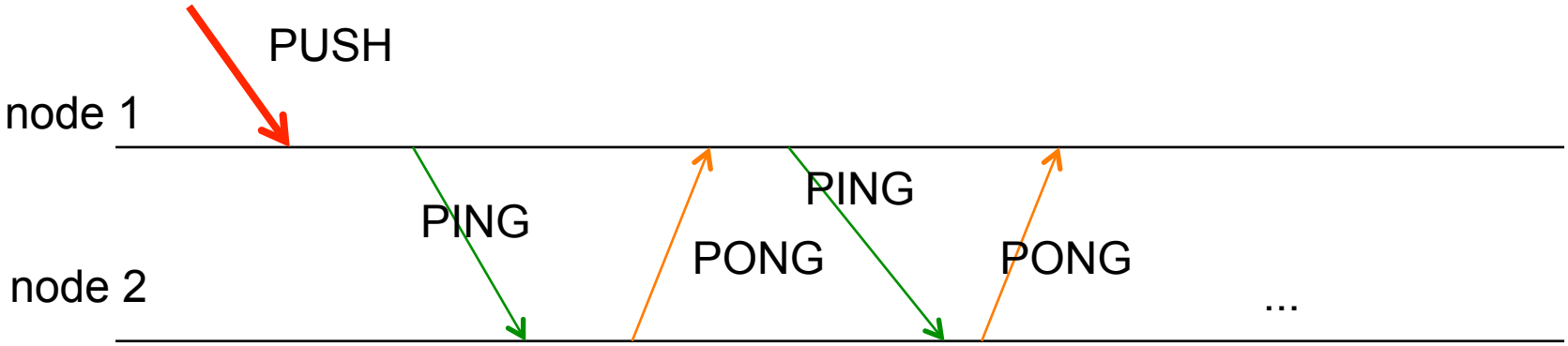
So, let us start with a simple example

- **The PING-PONG exchange**
  - two processes (nodes)
  - each one has a 'PUSH' button
  - when I push the button of one process, it sends 'PING' to the other
  - one the other recieves 'PING', it replies 'PONG'
  - when one receive 'PONG', it replies 'PING'
  - ☺



Time diagram





Let us define the communication interfaces

- **The communication interfaces correspond to the communication channels (the links)**

```
Client_itf REQ; //the interface to the outside
Comm_itf in, out; //ingoing and outgoing channel
```

Let us define the types of messages we will need

```
Message_t PUSH;
Message_t PING;
Message_t PONG;

//this is all pseudo code
```

Let's go ...

- **When the PUSH button is pushed, send PING**

```
REQ ? PUSH => out ! PING
```

- **When PING is received, send PONG**

```
in ? PING => out ! PONG
```

- **When PONG is received, send PING**

```
in ? PONG => out ! PING
```

EASY PEASY

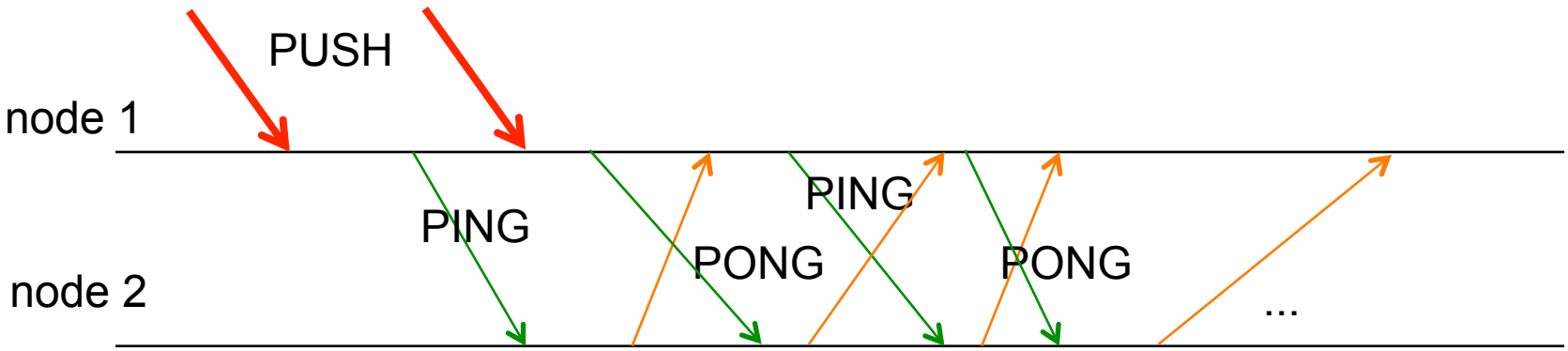
Let us think more about it...

- **Does this algorithms allow to push the button more than once?**

Let us think more about it...

- **Does this algorithms allow to push the button more than once?**

- **Well, YES ☹**



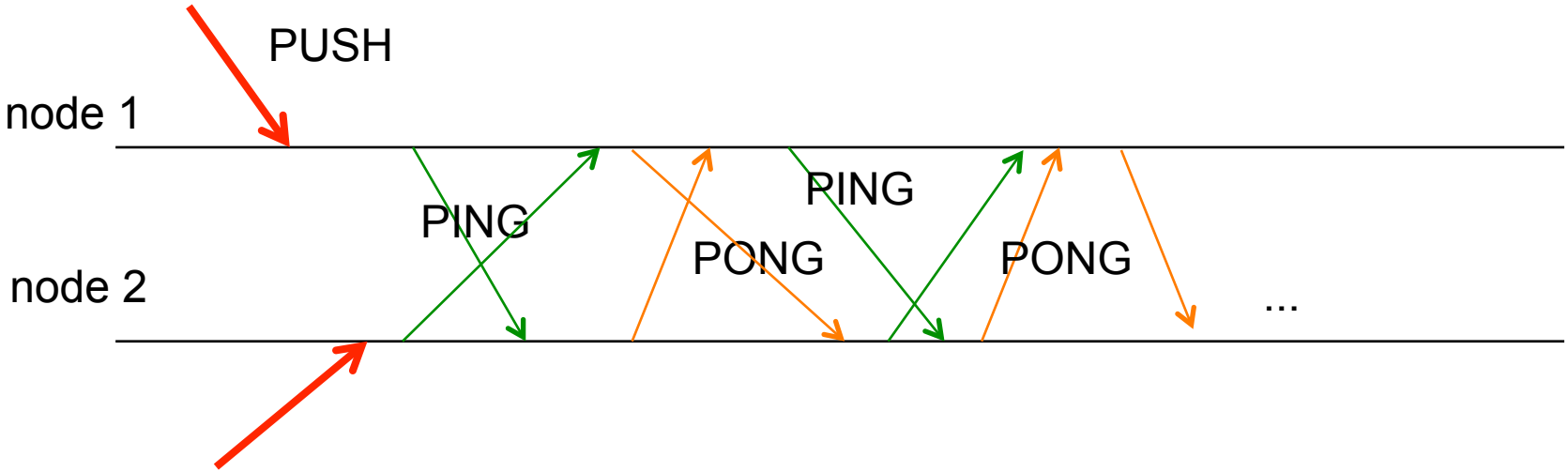
Let us think more about it...

- **Does this algorithms prevent us from pushing the buttons on both sides ?**

Let us think more about it...

- **Does this algorithms prevent us from pushing the buttons on both sides ?**

- **Well, NO ☹**



Let us prevent pushing the button more than once on one node

- **We will need a boolean to indicate whether the button has been already pushed or not**

```
boolean pushed = false
```

Let us integrate it into the algorithm

```
// Receive PUSH and button not pushed
REQ ? PUSH && !pushed =>   out ! PING
                           pushed = true;

// no rule for REQ?PUSH && pushed
// so nothing happens

in ? PING => out ! PONG

in ? PONG => out ! PING
```

For the second issue

- **Two cases**
  - Only one node starts and informs the other that it has started so that the other disables its PUSH button
  - Both nodes start "at the same time" and need to agree on who will start with the PING
    - to decide, we will use the node IDs
    - that are unique
  - ... a kind of "initiate session" phase
  - ... those who did the OS project with the network aspects will remember
  - to decide we will use

The algorithm

```
Message_t PUSH;
Message_t ASK_START, ACK_START, PING, PONG;

boolean started = false; //instead of pushed

// receive PUSH and node has not started
REQ ? PUSH && !started =>  out ! <ASK_START,ID>
                           started = true;

// receive ASK_START and node not started
in ? <ASK_START, id> && !started =>  out!ACK_START
                                     started = true;

//receive ASK_START but node has started, smaller id wins
in ? <ASK_START,id> && started && ID > id => out!ACK_START

in ? ACK_START => out!PING
in ? PING => out ! PONG
in ? PONG => out ! PING
```

The algorithm

```
Message_t PUSH;
Message_t ASK_START, ACK_START, PING, PONG;

boolean started = false; //instead of pushed

// receive PUSH and node has not started
REQ ? PUSH && !started =>  out ! <ASK_START,ID>
                           started = true;

// receive ASK_START and node not started
in ? <ASK_START, id> && !started =>  out!ACK_START
                                     started = true;

//receive ASK_START but node has started, smaller id wins
in ? <ASK_START,id> && started && ID > id => out!ACK_START

in ? ACK_START => out!PING
in ? PING => out ! PONG
in ? PONG => out ! PING
```

The algorithm

```
Message_t PUSH;
Message_t ASK_START, ACK_START, PING, PONG;

boolean started = false; //instead of pushed

// receive PUSH and node has not started
REQ ? PUSH && !started =>  out ! <ASK_START,ID>
                           started = true;

// receive ASK_START and node not started
in ? <ASK_START, id> && !started =>  out!ACK_START
                                     started = true;

//receive ASK_START but node has started, smaller id wins
in ? <ASK_START,id> && started && ID > id => out!ACK_START

in ? ACK_START => out!PING
in ? PING => out ! PONG
in ? PONG => out ! PING
```

The algorithm

```
Message_t PUSH;
Message_t ASK_START, ACK_START, PING, PONG;

boolean started = false; //instead of pushed

// receive PUSH and node has not started
REQ ? PUSH && !started =>  out ! <ASK_START,ID>
                           started = true;

// receive ASK_START and node not started
in ? <ASK_START, id> && !started =>  out!ACK_START
                                     started = true;

//receive ASK_START but node has started, smaller id wins
in ? <ASK_START,id> && started && ID > id => out!ACK_START

in ? ACK_START => out!PING
in ? PING => out ! PONG
in ? PONG => out ! PING
```

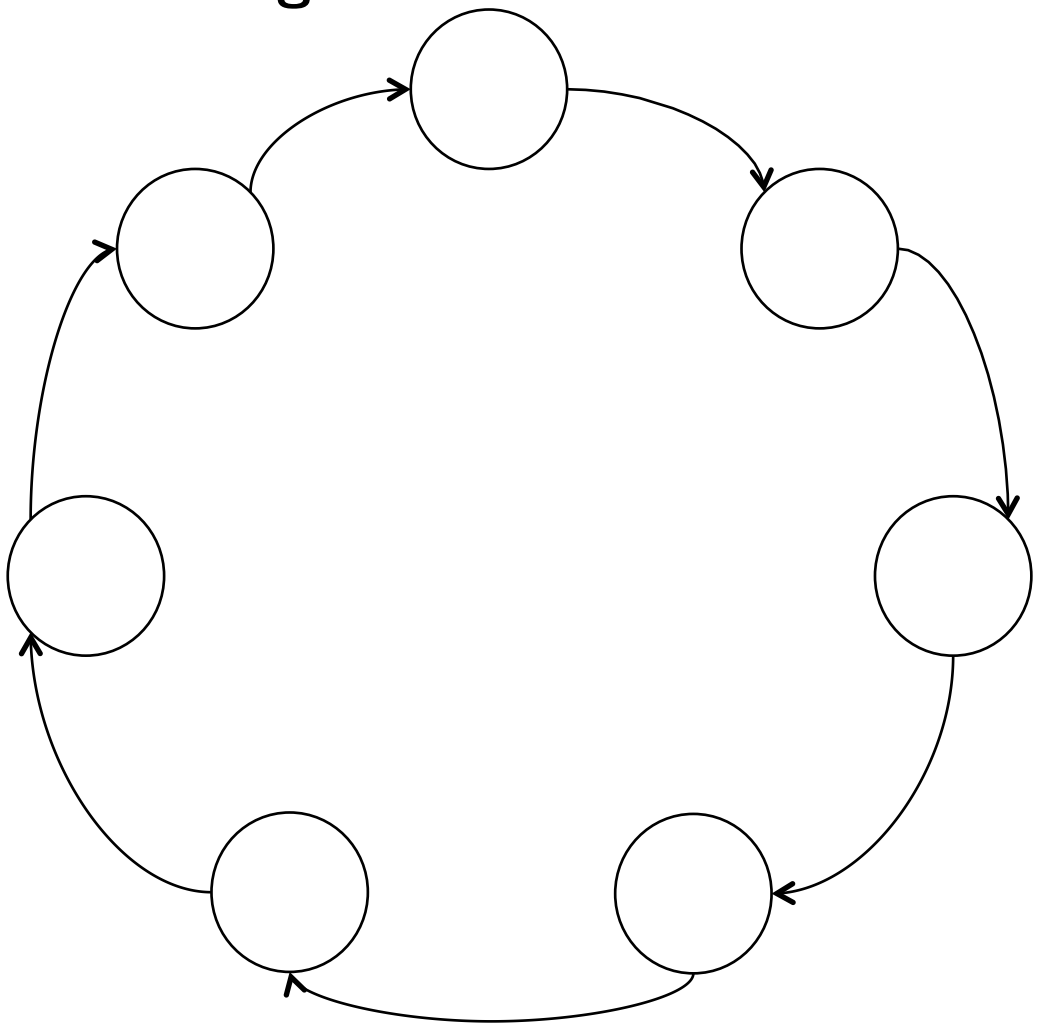
Wave Algorithm

- **Algorithm in which all processes need to do something before one node (the initiator) decides of something**
- **General Specification**

  An algorithm is a wave algorithm if it satisfies the following properties
  - its execution is finite (the algo terminates)
  - there is one decision event
  - the decision is preceded (to be seen in the next lecture) by at least one event at each node
- **Data Broadcast Instance**
  - an initiator starts to broadcast a message
  - all nodes receive the message
  - the initiator is notified that the nodes have received the message
  - in finite time

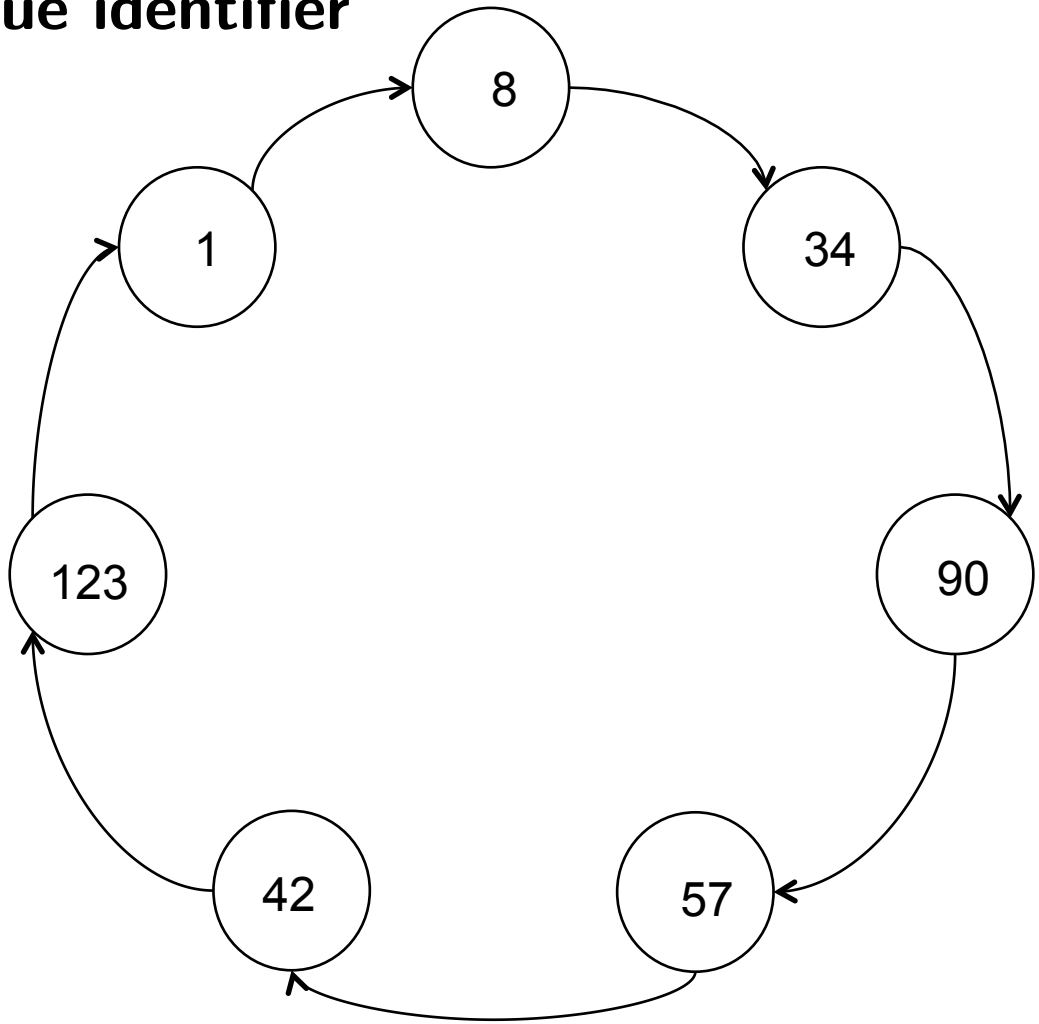
Data Broadcast on a Ring

- **Unidirectional ring**



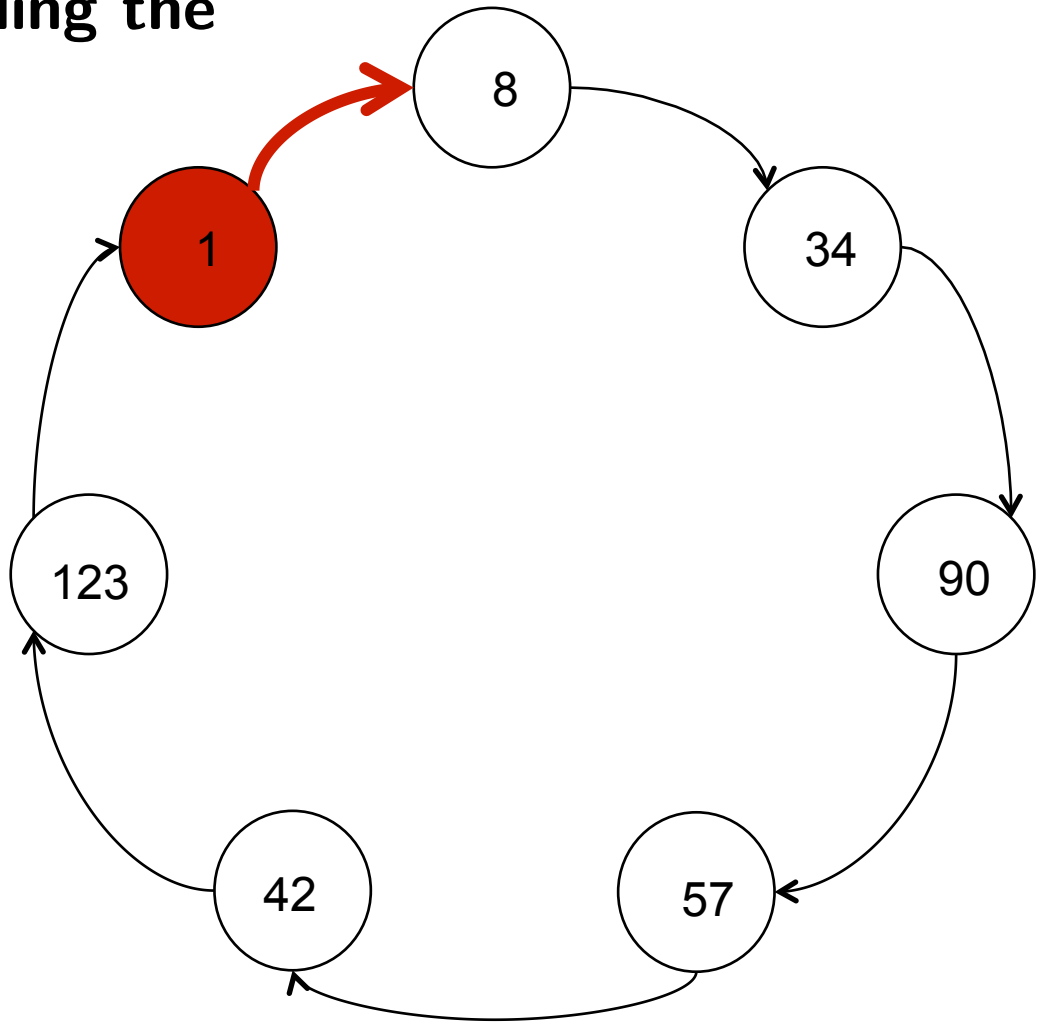
Data Broadcast on a Ring

- **All nodes have an unique identifier**



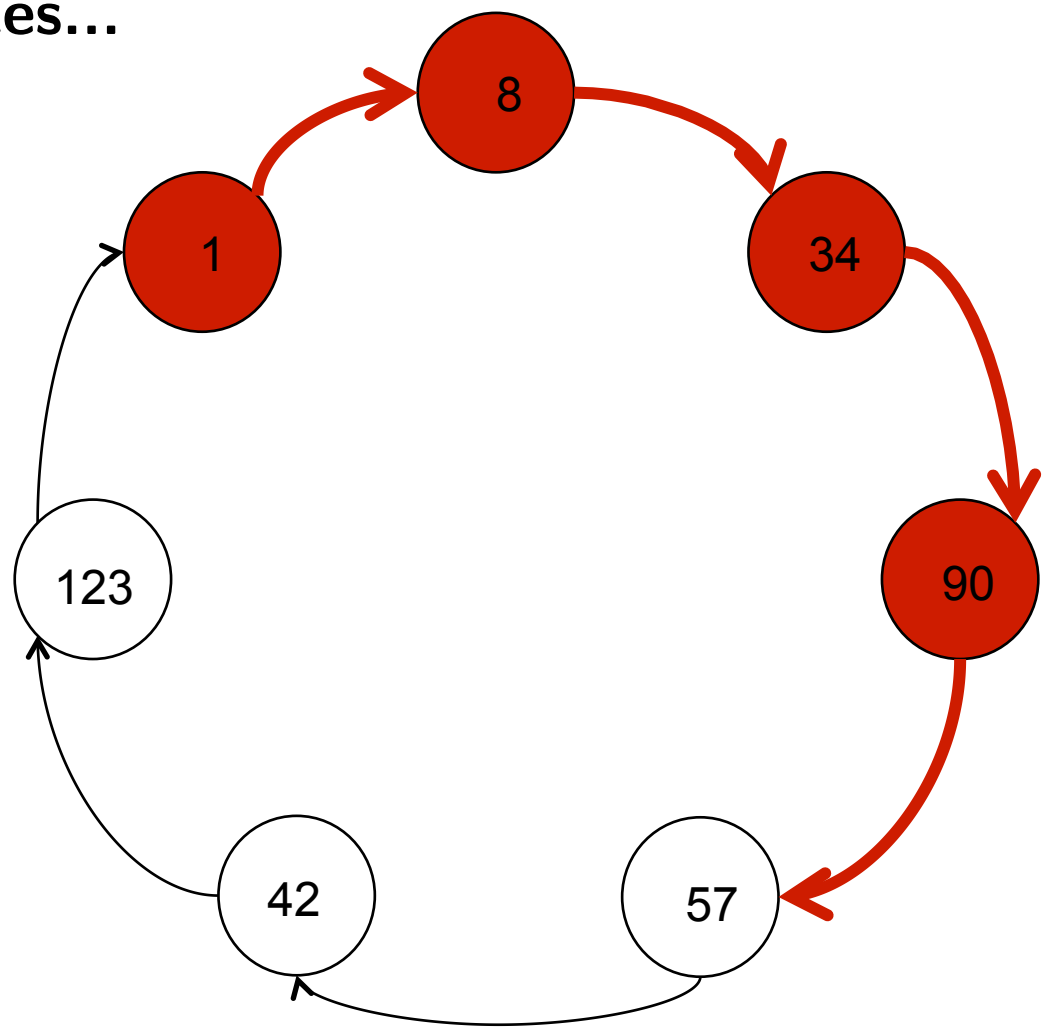
Data Broadcast on a Ring

- **An initiator starts sending the data**



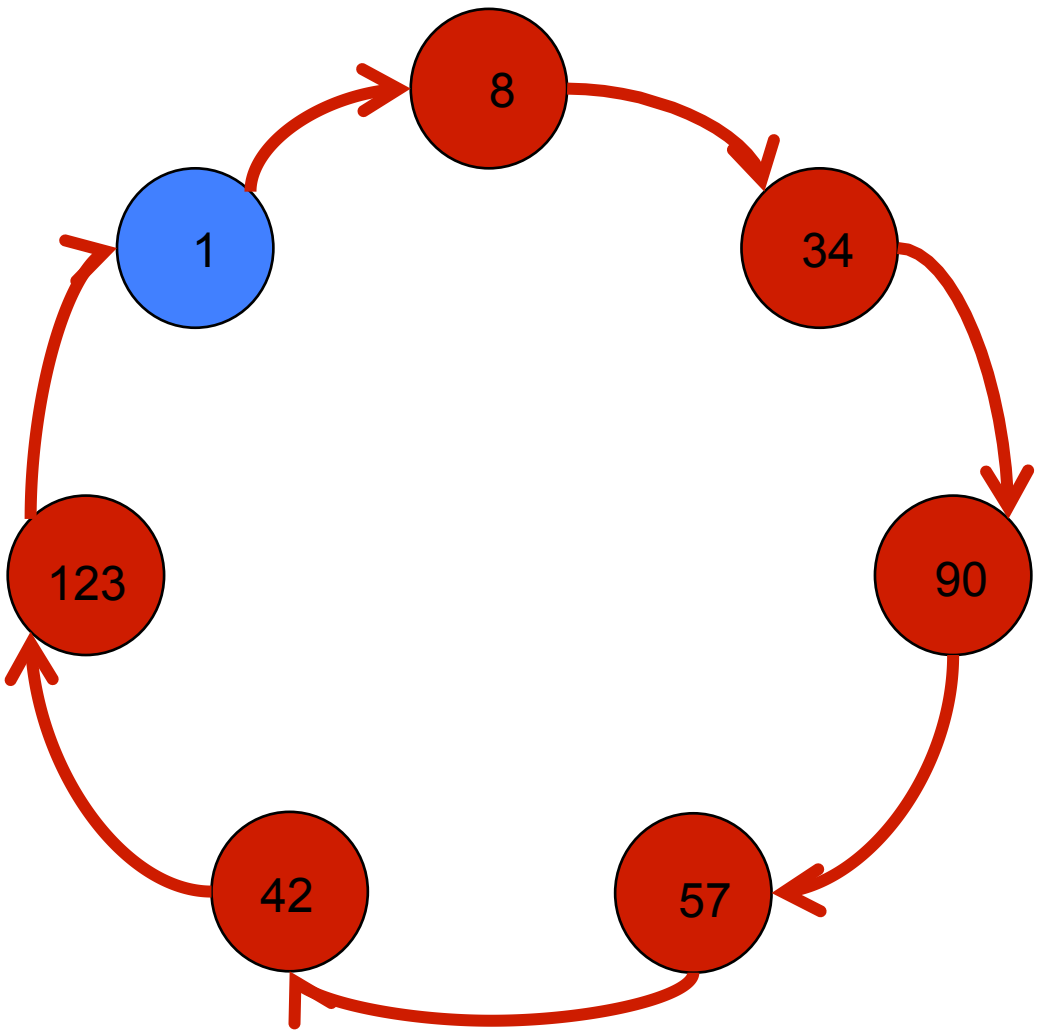
Data Broadcast on a Ring

- **The message propagates...**



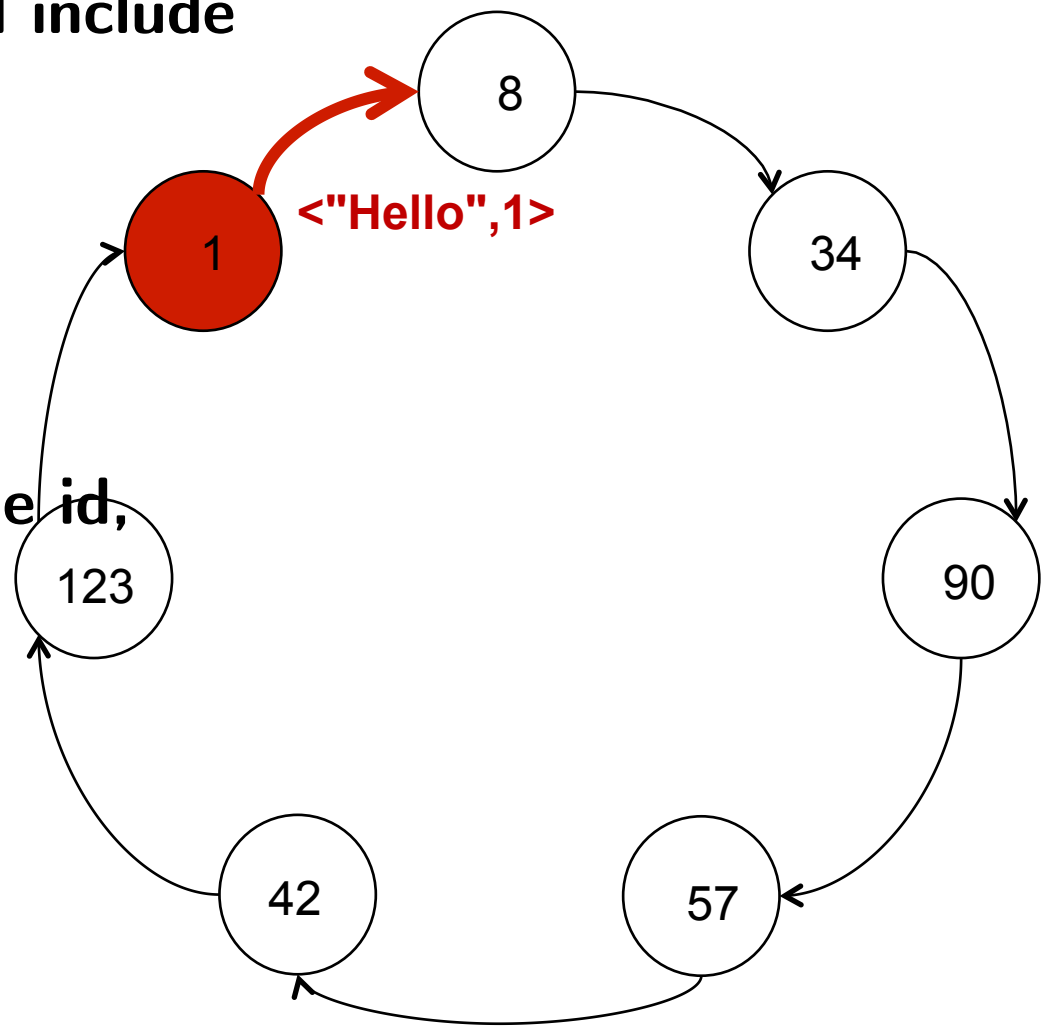
Data Broadcast on a Ring

- **and should stop when getting back to the initiator**



Data Broadcast on a Ring

- **So the message should include the ID of the node**
- **when it is on a node with different ID, it should proceed**
- **when it arrives at the node with the same id, it needs to stop**



//this is an algo we
//already talked about in
//the last lecture

Data Broadcast on a Ring

```
Client_itf REQ;   //the interface to the outside
Comm_itf in, out; //ingoing and outgoing channel

Message_t <START, Payload_t MSG>;
Message_t <FLOOD,Payload_t MSG,int ID>;

//rule 1 : initiator starting
REQ ? <START, msg> => out ! <FLOOD, msg, ID>

//rule 2 : transfer
in ? <FLOOD, msg,id> && id != ID => out ! <FLOOD,msg,id>
//another way to write it
//in ? Message_t M = <FLOOD, msg,id> && id != ID => out ! M

//rule 3 : arrive at the initiater after a full tour
in ? <FLOOD, msg,id> && id == ID =>  callback_client
                                     //TERMINATE
```

Data Broadcast on a Ring

```
Client_itf REQ;   //the interface to the outside
Comm_itf in, out; //ingoing and outgoing channel

Message_t <START, Payload_t MSG>;
Message_t <FLOOD,Payload_t MSG,int ID>;

//rule 1 : initiator starting
REQ ? <START, msg> => out ! <FLOOD, msg, ID>

//rule 2 : transfer
in ? <FLOOD, msg,id> && id != ID => out ! <FLOOD,msg,id>
//another way to write it
//in ? Message_t M = <FLOOD, msg,id> && id != ID => out ! M

//rule 3 : arrive at the initiater after a full tour
in ? <FLOOD, msg,id> && id == ID =>  callback_client
                                     //TERMINATE
```

Data Broadcast on a Ring

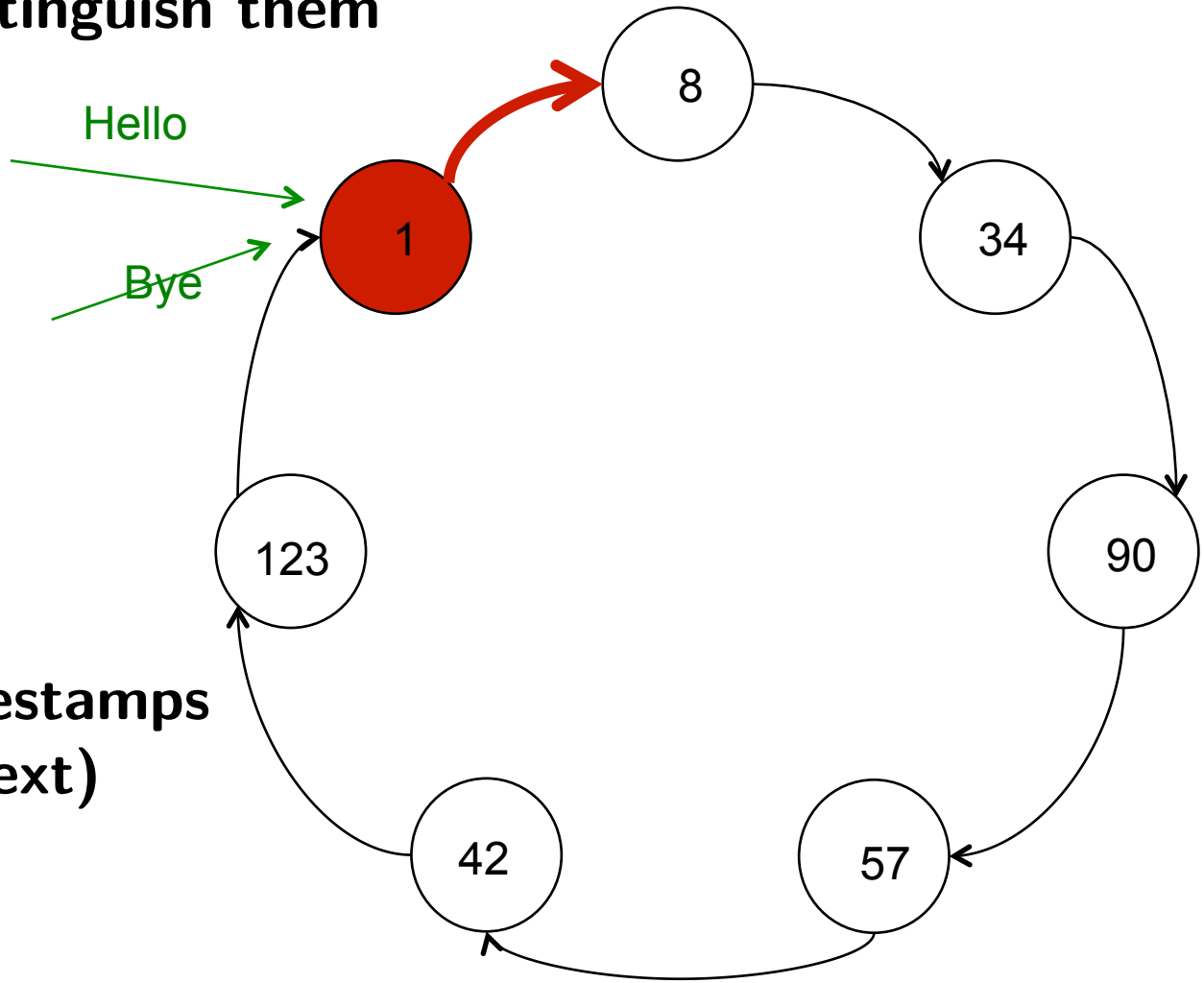
```
Client_itf REQ;   //the interface to the outside
Comm_itf in, out; //ingoing and outgoing channel

Message_t <START, Payload_t MSG>;
Message_t <FLOOD,Payload_t MSG,int ID>;

//rule 1 : initiator starting
REQ ? <START, msg> => out ! <FLOOD, msg, ID>

//rule 2 : transfer
in ? <FLOOD, msg,id> && id != ID => out ! <FLOOD,msg,id>
//another way to write it
//in ? Message_t M = <FLOOD, msg,id> && id != ID => out ! M

//rule 3 : arrive at the initiater after a full tour
in ? <FLOOD, msg,id> && id == ID =>  callback_client
                                     //TERMINATE
```

Data Broadcast on a Ring

```
Client_itf REQ;   //the interface to the outside
Comm_itf in, out; //ingoing and outgoing channel

Message_t <START, Payload_t MSG>;
Message_t <FLOOD,Payload_t MSG,int ID>;

//rule 1 : initiator starting
REQ ? <START, msg> => out ! <FLOOD, msg, ID>

//rule 2 : transfer
in ? <FLOOD, msg,id> && id != ID => out ! <FLOOD,msg,id>
//another way to write it
//in ? Message_t M = <FLOOD, msg,id> && id != ID => out ! M

//rule 3 : arrive at the initiater after a full tour
in ? <FLOOD, msg,id> && id == ID =>  callback_client
                                     //TERMINATE
```

How do we manage multiple requests?

- **We need to distinguish them**



- **Number them (related to timestamps we will study next)**

Data Broadcast on a Ring

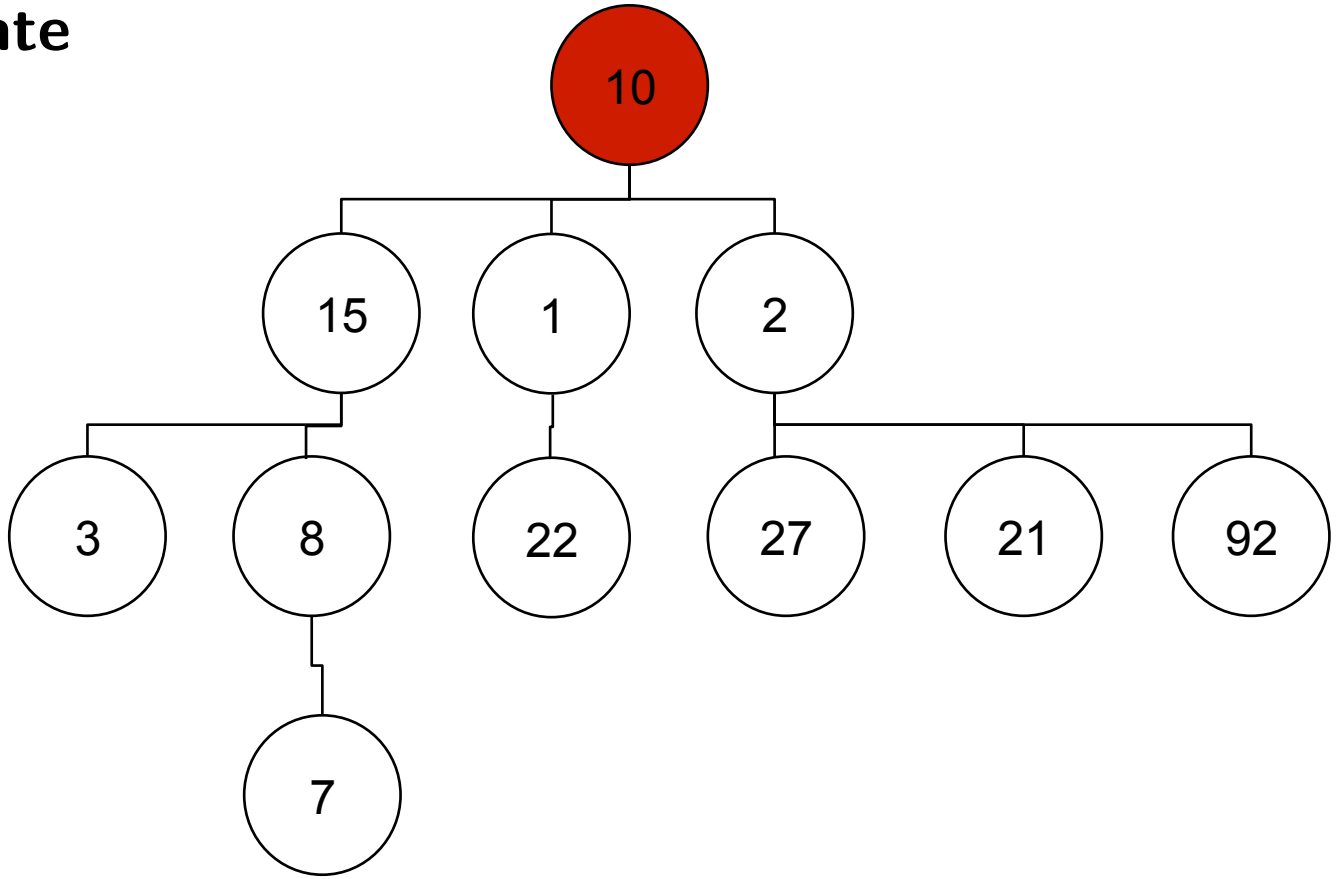
```
Client_itf REQ;   //the interface to the outside
Comm_itf in, out; //ingoing and outgoing channel
Message_t <START, Payload_t MSG>;
Message_t <FLOOD,Payload_t MSG,int ID, int N>;

int N=0 //to number the requests

//rule 1 : initiator starting
REQ ? <START, msg> =>  out ! <FLOOD, msg, ID, N>
                       N++;

//rule 2 : transfer
in ? <FLOOD, msg,id, n> && id != ID => out ! <FLOOD,msg,id, n>
//another way to write it
//in ? Message_t M = <FLOOD, msg,id, n> && id != ID => out ! M

//rule 3 : arrive at the initiater after a full tour
in ? <FLOOD, msg,id, n> && id == ID =>  callback_client for
                                        request n
                                        //TERMINATE
```

Data Broadcast on a Ring

- **Does the algo work if there are multiple initiators?**

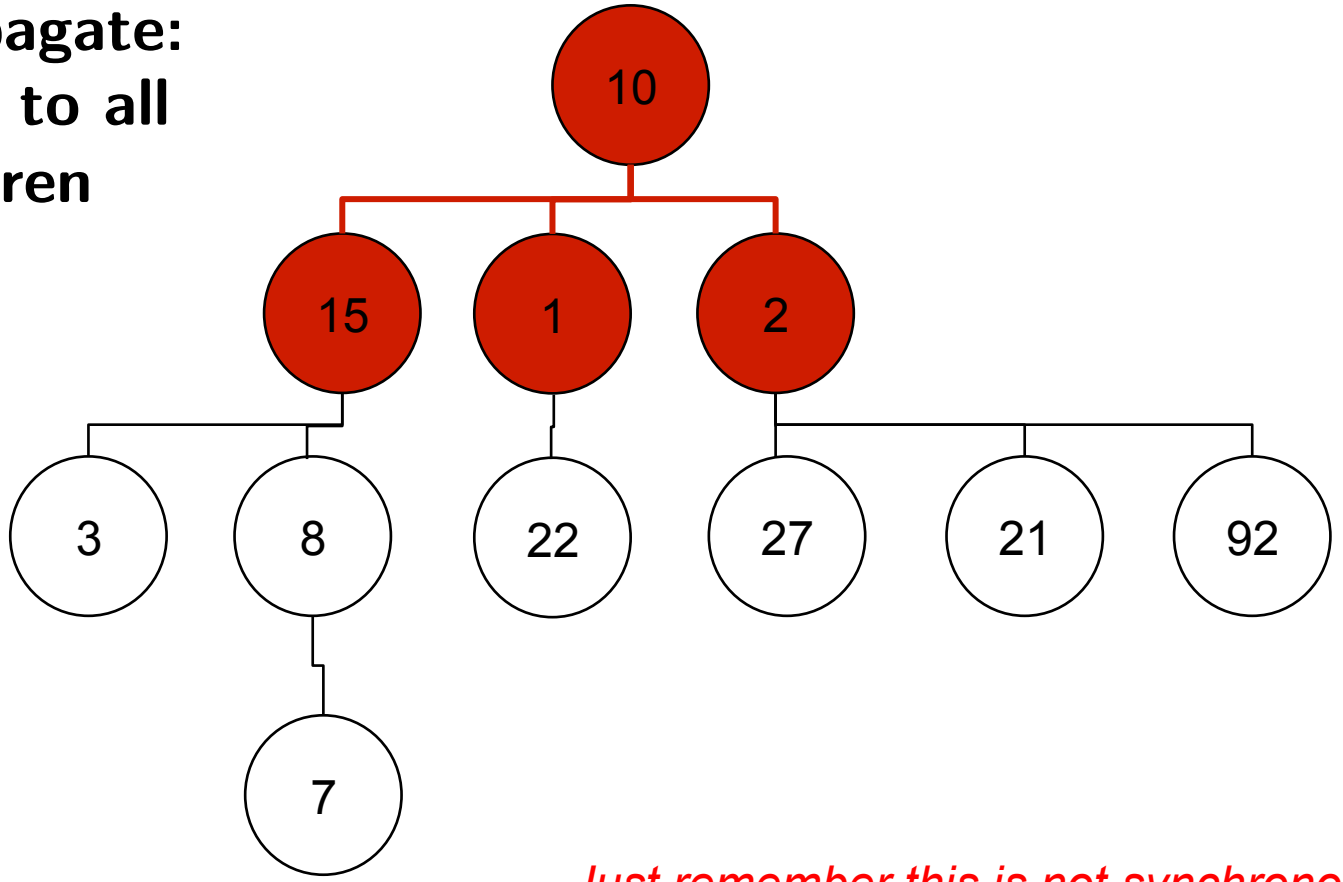
Data Broadcast on a Tree

- **Initiate**



Data Broadcast on a Tree

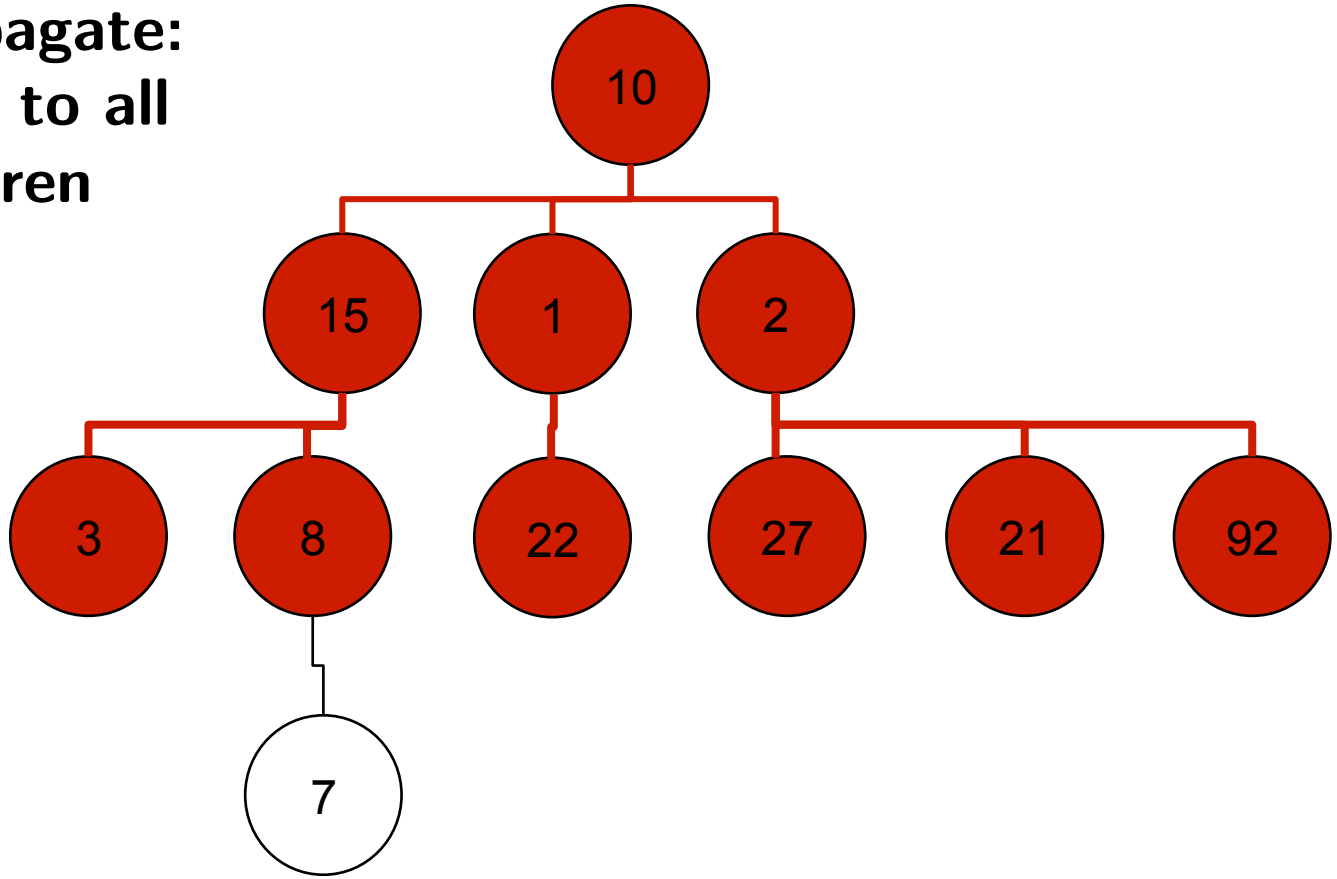
- **Propagate: send to all children**



Just remember this is not synchronous!

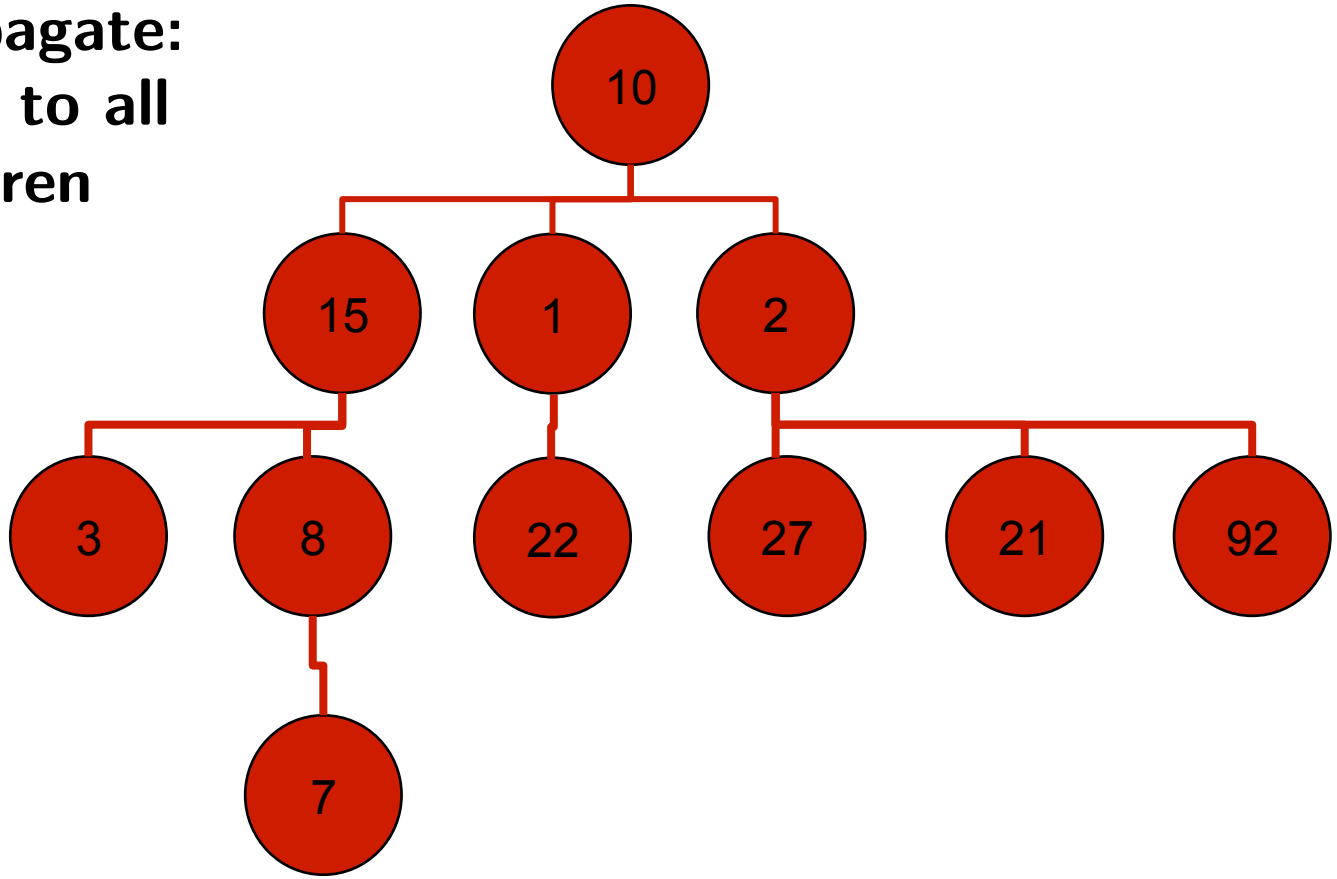
Data Broadcast on a Tree

- **Propagate: send to all children**



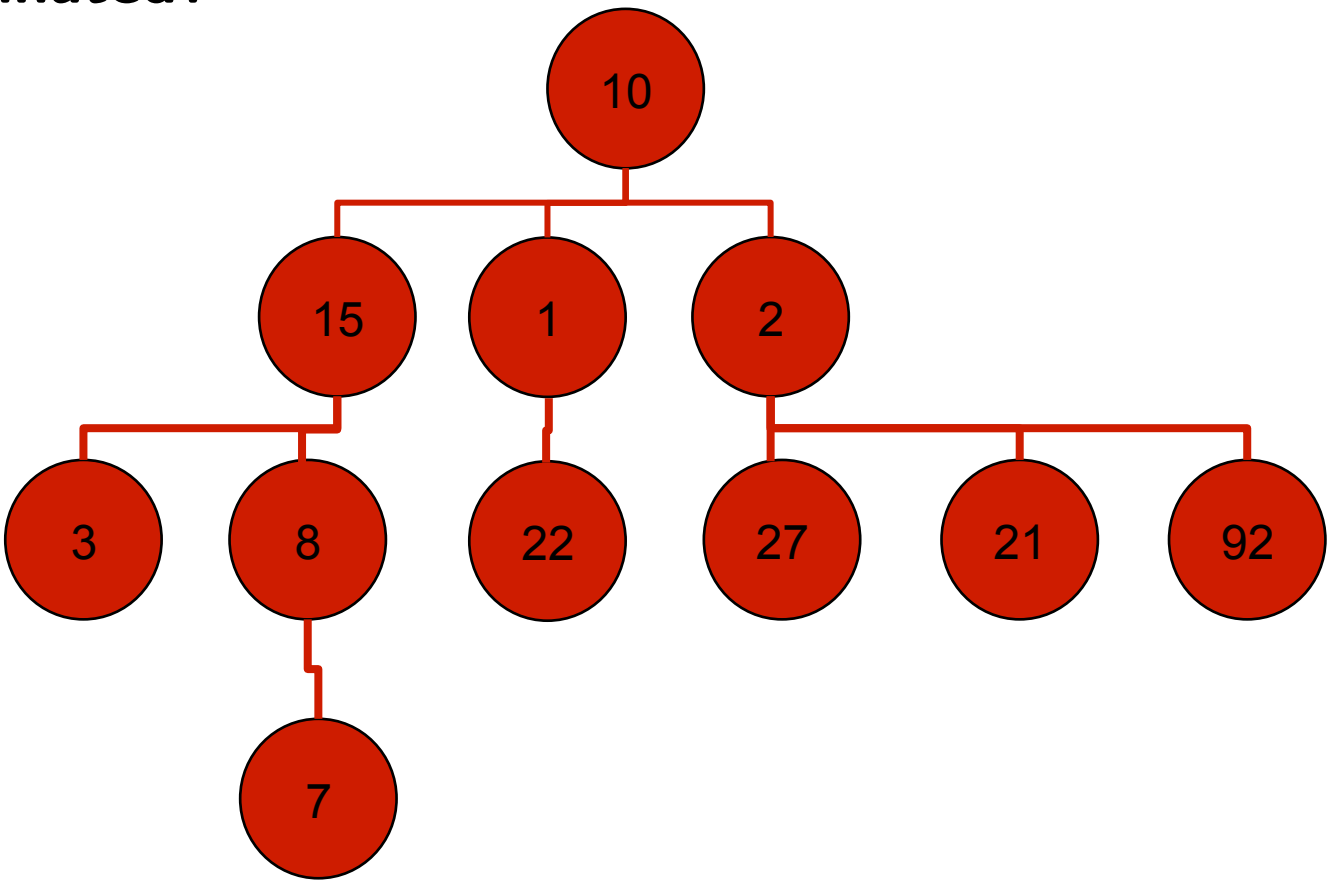
Data Broadcast on a Tree

- **Propagate: send to all children**



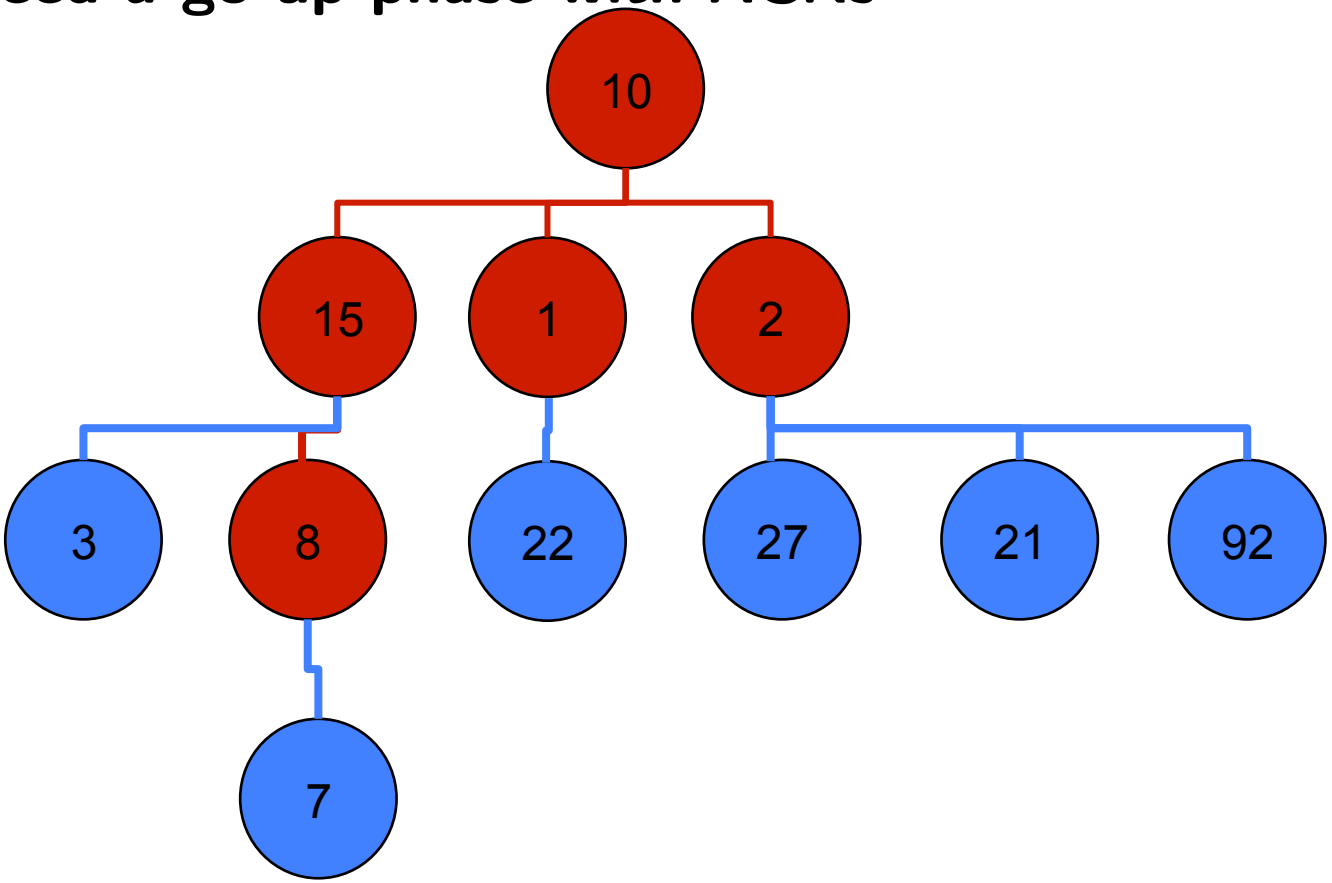
Data Broadcast on a Tree

- **Question : how does the initiator know that it is terminated?**



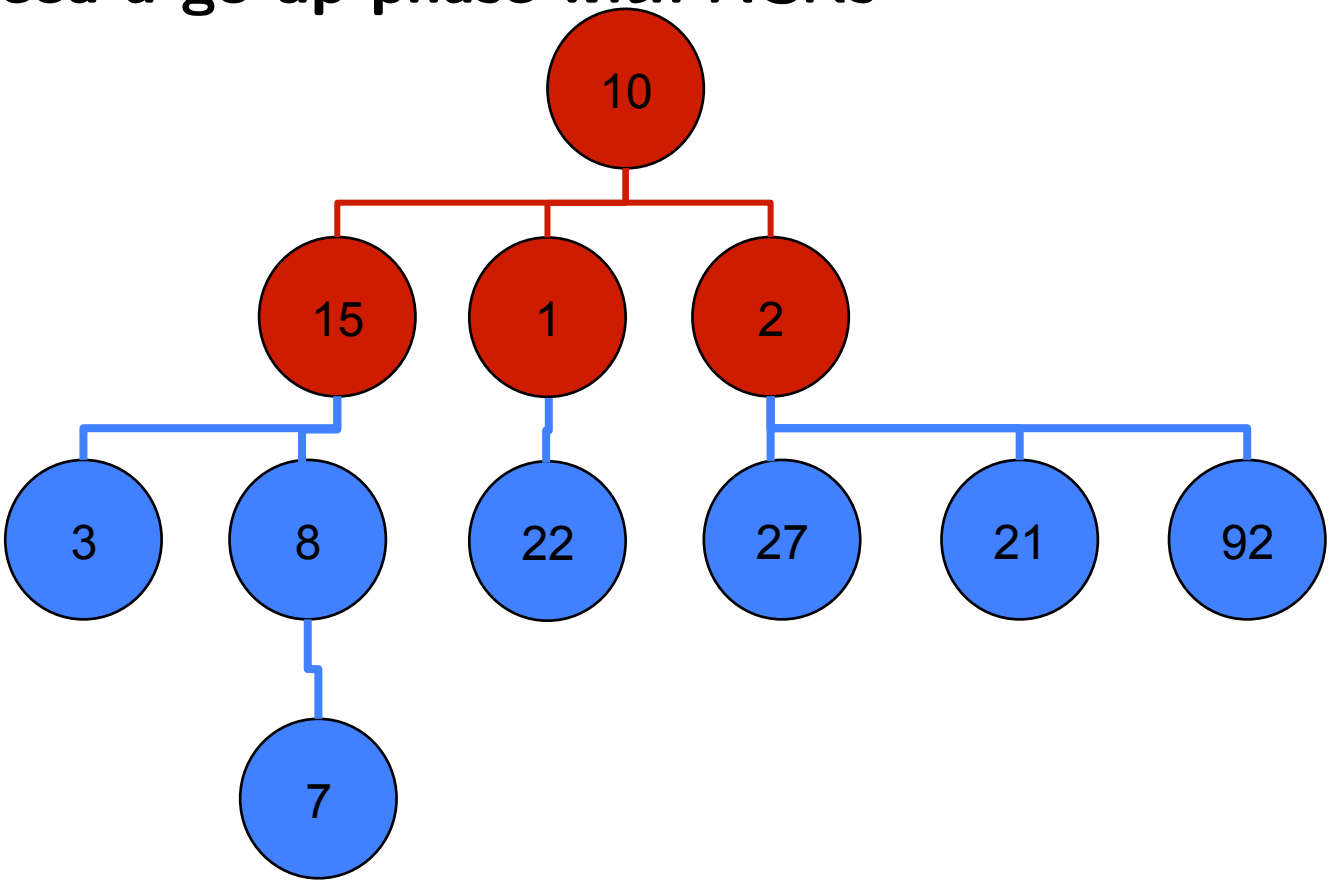
Data Broadcast on a Tree

- **If the data propagation is a go-down phase, we need a go-up phase with ACKs**



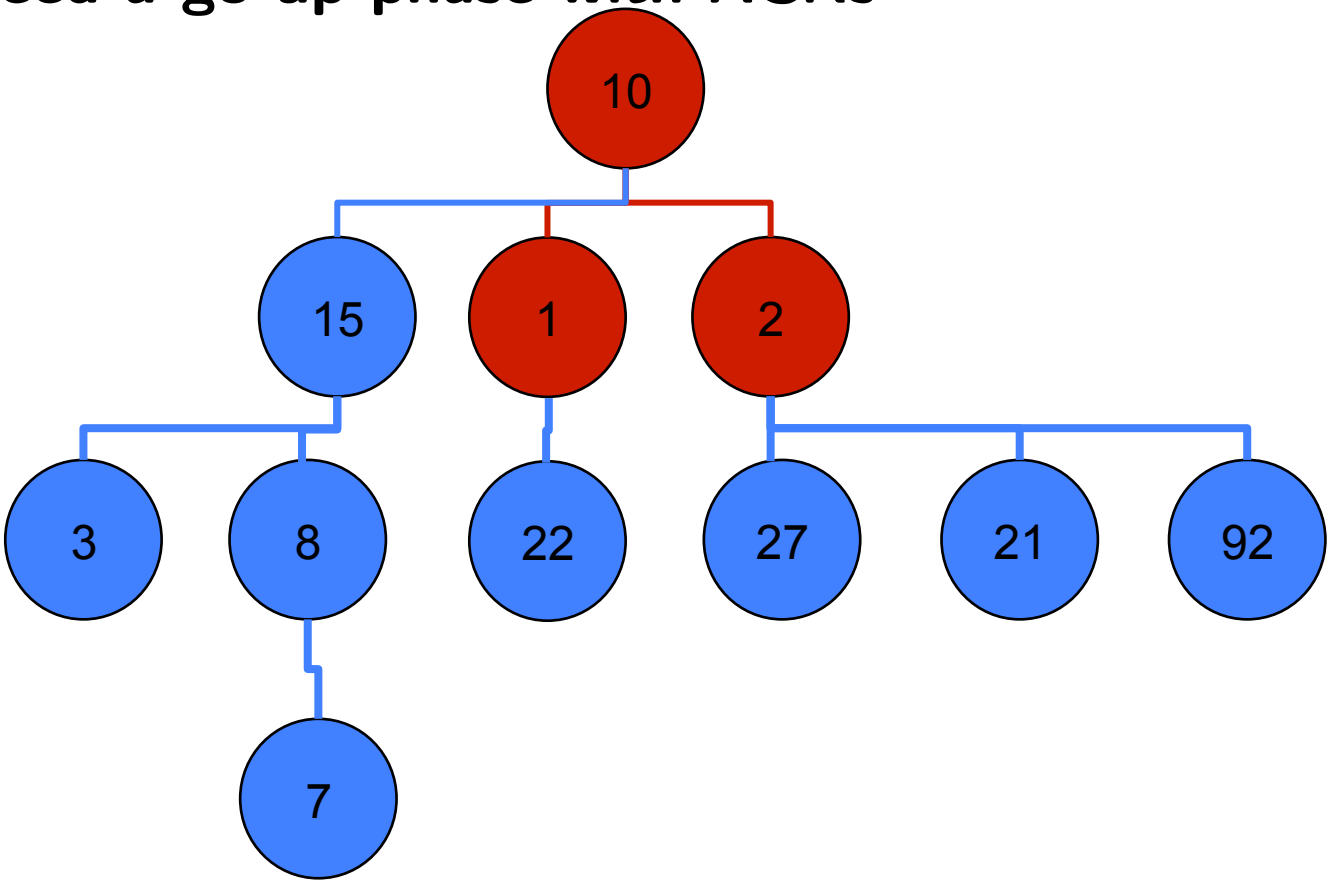
Data Broadcast on a Tree

- **If the data propagation is a go-down phase, we need a go-up phase with ACKs**



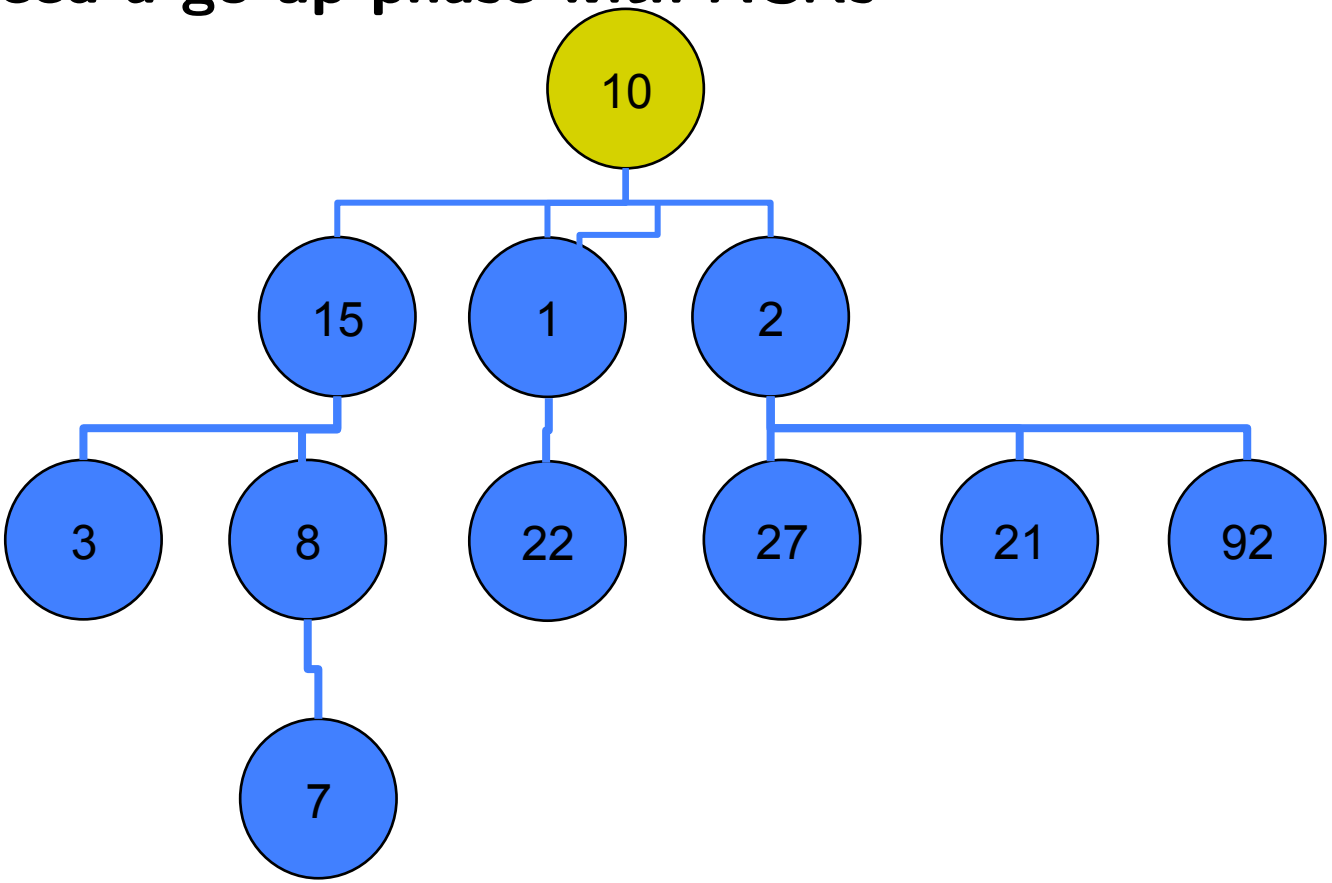
Data Broadcast on a Tree

- **If the data propagation is a go-down phase, we need a go-up phase with ACKs**



Data Broadcast on a Tree

- **If the data propagation is a go-down phase, we need a go-up phase with ACKs**



Algorithm (one initiator, one wave at a time)

```
Client_itf REQ;              //the interface to the outside
CommSet_itf children;        //set of channels
Message_t <START, Payload_t MSG>;
Message_t <FLOOD,Payload_t MSG, int ID>, <ACK, int ID>;
int nb_ack=0;
Comm_itf up;

REQ ? <START, msg> =>        forall s in S, s!<FLOOD,msg, ID>
                             nb_ack=0;
//go down, arrive at leaf
exists s in S, s?<FLOOD, msg, id> && |S| == 1 => s!<ACK,id>
//go down, arrive at not leaf
exists s in S, s?<FLOOD, msg, id> && |S| > 1 =>
              for all s' in S\{s}, s'! <FLOOD, msg, id>
              nb_ack=0; up = s;
```

Algorithm (one initiator, one wave at a time)

```
Client_itf REQ;              //the interface to the outside
CommSet_itf children;        //set of channels
Message_t <START, Payload_t MSG>;
Message_t <FLOOD,Payload_t MSG, int ID>, <ACK, int ID>;
int nb_ack=0;
Comm_itf up;

REQ ? <START, msg> =>        forall s in S, s!<FLOOD,msg, ID>
                             nb_ack=0;
//go down, arrive at leaf
exists s in S, s?<FLOOD, msg, id> && |S| == 1 => s!<ACK,id>
//go down, arrive at not leaf
exists s in S, s?<FLOOD, msg, id> && |S| > 1 =>
              for all s' in S\{s}, s'! <FLOOD, msg, id>
              nb_ack=0; up = s;
```

Algorithm (one initiator, one wave at a time)

```
Client_itf REQ;              //the interface to the outside
CommSet_itf children;        //set of channels
Message_t <START, Payload_t MSG>;
Message_t <FLOOD,Payload_t MSG, int ID>, <ACK, int ID>;
int nb_ack=0;
Comm_itf up;

REQ ? <START, msg> =>        forall s in S, s!<FLOOD,msg, ID>
                             nb_ack=0;
//go down, arrive at leaf
exists s in S, s?<FLOOD, msg, id> && |S| == 1 => s!<ACK,id>
//go down, arrive at not leaf
exists s in S, s?<FLOOD, msg, id> && |S| > 1 =>
              for all s' in S\{s}, s'! <FLOOD, msg, id>
              nb_ack=0; up = s;
```

Algorithm (one initiator, one wave at a time)

```
Client_itf REQ;              //the interface to the outside
CommSet_itf children;        //set of channels
Message_t <START, Payload_t MSG>;
Message_t <FLOOD,Payload_t MSG, int ID>, <ACK, int ID>;
int nb_ack=0;
Comm_itf up;

REQ ? <START, msg> =>        forall s in S, s!<FLOOD,msg, ID>
                             nb_ack=0;
//go down, arrive at leaf
exists s in S, s?<FLOOD, msg, id> && |S| == 1 => s!<ACK,id>
//go down, arrive at not leaf
exists s in S, s?<FLOOD, msg, id> && |S| > 1 =>
              for all s' in S\{s}, s'! <FLOOD, msg, id>
              nb_ack=0; up = s;
```

Algorithm (one initiator, one wave at a time)

```
//go up, ack arrives at node that is not initiator
exists s in S, s?<ACK,id> && id != ID =>
                              nb_ack++;
                              if nb_ack == |S|-1, up!ACK

//go up, ack arrives at node is the initiator
exists s in S, s?<ACK,id> && id == ID =>
                              nb_ack++;
                              if nb_ack == |S|-1, callback_client
                                     //TERMINATE
```

Algorithm (one initiator, one wave at a time)

```
//go up, ack arrives at node that is not initiator
exists s in S, s?<ACK,id> && id != ID =>
                              nb_ack++;
                              if nb_ack == |S|-1, up!ACK

//go up, ack arrives at node is the initiator
exists s in S, s?<ACK,id> && id == ID =>
                              nb_ack++;
                              if nb_ack == |S|-1, callback_client
                                    //TERMINATE
```

Broadcast on a Tree

- **Can you finish the algo for it to be**
  - capable of managing multiple initiators
  - multiple waves?

Broadcast on a Graph

- **What is the difference between a tree and a graph?**

Broadcast on a Graph

- **What is the difference between a tree and a graph?**
  - In a graph there may be **loops**.

- **Modify the tree algorithms so as to manage loops.**